

Achilles RAG Eğitimi — Detaylı Anlatım

Sürüm: v1.1 · 2026-06-16

Sürüm Geçmişi

Sürüm	Tarih	Değişiklik
v1.0	2026-06-16	İlk kapsamlı sürüm.
v1.1	2026-06-16	Denetim düzeltmeleri: embedding boyutu (256-d yalnız fake yedek; nomic-embed-text boyutu Ollama modeli tarafından belirlenir), chunk_size/overlap doğru satır referansı (settings.py:77-78), hayali "~8000 chunk_size çelişkisi" kaldırıldı, test kapsamı doğru yansıtıldı (BM25/cross-encoder ve L3/L4/L5 testleri mevcut), auto-chain eğitim çağrısı doğru tarif edildi (doğrudan CLI, web subprocess değil).

Not: Yeni eğitim geliştirmesinde sürüm numarası artırılır ve değişiklik buraya eklenir.

Amaç ve Kapsam

Bu doküman, Achilles araştırma sisteminin **RAG (Retrieval-Augmented Generation)** eğitim hattını uçtan uca, dosya:satır referansıyla anlatır. Achilles, CLAUDE.md'de tanımlandığı gibi **yerel-öncelikli bir AI trading araştırma sistemidir**: canlı bot değil, yatırım tavsiyesi değil. Çıktılar her zaman *hipotez + test noktası* biçimindedir.

Kapsam, aşağıdaki bileşenlerin tamamını içerir:

- Ingestion + chunking** — PDF metin çıkarma, matematik-farkında parçalama (chunker.py, hybrid_chunker.py).
- Embedding + vektör depo** — nomic-embed-text (Ollama) + ChromaDB (embedding_service.py, chroma_store.py).
- Retrieval** — hibrit BM25 + dense, over-fetch + cross-encoder/heuristik rerank, contextual retrieval, query genişletme (reranking_retriever.py, bm25_index.py, query_expander.py).
- Bilgi kartı + doğrulama** — citation/grounding/abstention, bağlam yeterliliği, güven skoru (knowledge_card_builder.py, verification/*).
- rag_mastery metriği** — coverage + comprehension + train_readiness bileşik skoru.
- Anlama Doğrulama Merdiveni** — Taban/L1/L2/L3/L4/L5 + UnderstandingScore, ENTROPY/PERMENTROPY göstergeleri, araştırma döngüsü (sentez → backtest → yansıma → L5).
- RAG ile ilgili loop'lar** — continuous-learning, auto-chain, mac-loop, detached eğitim.

Önemli dürüstlük notu (CLAUDE.md Kural 2): Bu doküman, bileşenlerin "çalıştığını" iddia etmez; yalnızca kodda **nasıl tanımlandıklarını** anlatır. Test durumu açıkça bilinmediğinde "test durumu kodda bulunamadı" yazılır. Aynı şekilde, beklenen ama kodda bulunamayan yapılar açıkça işaretlenir.

Mimari Genel Bakış

ACHILLES RAG EĞİTİM HATTI

```
graph TD
    Input["[PDF Dosyaları]  
raw_pdf/"] --> P1["pdf_parser.py  
PyMuPDF→pypdf"]
    P1 -- "parse_pdf()  
ParsedPdf(pages)" --> P2["extract_metadata()  
başlık/yazar/yıl"]
    P1 -- "chunk_parsed_pdf()  
chunker.py / hybrid_chunker  
matematik-farkında parçalar  
($$..$$, \\begin{equation})" --> P3["list[TextChunk]"]
    P2 -- "[İDEMPOTENCY KONTROLÜ]  
file_hash + title dedup" --> P3
    P3 --> P4["SQLite store  
papers, chunks,  
formulas, cards"]
    P3 --> P5["build_embed_text()  
(contextual prefix)"]
    P5 --> P6["EmbeddingService.embed()  
Ollama nomic-embed-text  
fallback: fake (SHA256)"]
    P6 --> P7["ChromaDB  
paper_chunks  
cosine / HNSW"]
    P7 --> P8["RETRIEVAL"]
    P8 --> P9["QueryExpander → MultiQuery (opt)  
RerankingRetriever:  
dense(overfetch×4) + BM25(opt)  
→ rerank(heuristik | cross-encoder)"]
    P9 --> Output["list[RetrievedChunk]"]
    P8 -.-> P10["POST-PROCESSING  
(hata bloklamaz)  
├ FormulaExtractor  
├ ConceptGraph  
└ CrossPaperSynthesizer"]
    P10 -.-> P4
```

The diagram illustrates the Achilles RAG training pipeline, which is divided into two main paths: data processing and retrieval.

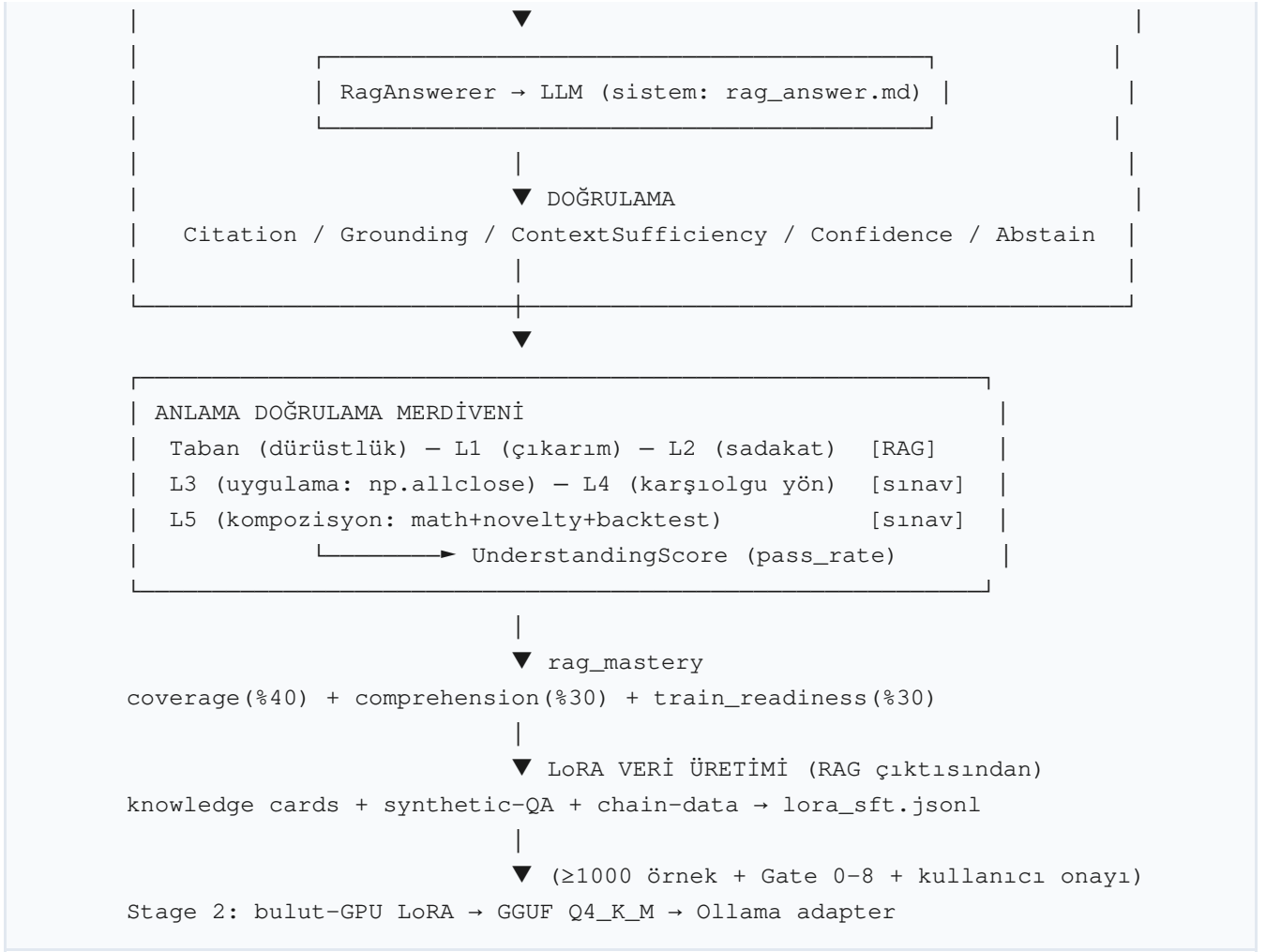
Data Processing Path:

- Input:** [PDF Dosyaları] raw_pdf/
- Processing:** pdf_parser.py (PyMuPDF→pypdf) processes the raw PDFs using parse_pdf() to create ParsedPdf(pages).
- Chunking:** chunk_parsed_pdf() uses chunker.py / hybrid_chunker to create chunks, specifically handling mathematical formulas like $\\begin{equation}$.
- Storage:** The chunks are stored in a SQLite database (papers, chunks, formulas, cards).
- Embedding:** build_embed_text() (contextual prefix) is used to create embeddings using EmbeddingService.embed() (Ollama nomic-embed-text, fallback: fake (SHA256)).
- Indexing:** The embeddings are stored in ChromaDB (paper_chunks, cosine / HNSW).

Retrieval Path:

- RETRIEVAL:** The retrieval process involves QueryExpander → MultiQuery (opt) and RerankingRetriever (dense(overfetch×4) + BM25(opt) → rerank(heuristik | cross-encoder)).
- Output:** list[RetrievedChunk]

Post-Processing: The retrieval process also involves POST-PROCESSING (hata bloklamaz) which includes FormulaExtractor, ConceptGraph, and CrossPaperSynthesizer.



Aşama Aşama Yöntemler

Aşama 1 — PDF Okuma ve Metin Çıkarımı

NE: PDF dosyasını sayfa sayfa düz metne dönüştürür.

NASIL: `parse_pdf(path)` önce PyMuPDF (fitz) ile `page.get_text("text")` çağırır; başarısız olursa pypdf yedek backend'ine (`page.extract_text()`) düşer. Her ikisi de yoksa `RuntimeError` fırlatır. Çıktı `ParsedPdf(path, pages: list[str])` dataclass'ıdır; `text`, `n_pages`, `n_chars` property'leri sağlar.

Dosya: `app/ingestion/pdf_parser.py:56-73`

Model/kütüphane: PyMuPDF (birincil), pypdf (yedek).

Aşama 2 — Matematik-Farkında Chunking

NE: Metni `chunk_size` sınırı içinde, paragraf-bilinçli parçalara böler; matematik bloklarını bölmeden korur.

NASIL (temel — `chunker.py`): Greedy packing. Buffer dolana kadar paragraf ekler; sığmazsa chunk üretir ve `overlap` kadar kuyruğu yeni buffer'a taşır. Tek paragraf çok büyükse:

- "matematik ağır" ise (`_is_math_heavy`) bölmeden **oversized** chunk olarak kabul eder;
- aksi halde nokta/satırsonu sınırından keser.

Matematik tespiti iki yolla:

- `_MATH_BLOCK_RE` regex'i: `$$...$$, \[...\], \begin{equation|align|gather|multline}`, satır içi `$. . . $` (3–120 karakter) — `chunker.py:24–30`.
- Karakter oranı: matematik karakteri (`$\d\S\P\I\sqrt{\leq\geq\neq\in}...` = vb.) oranı > 0.12 ise `math-heavy` — `chunker.py:31–39`.

NASIL (gelişmiş — `hybrid_chunker.py`): Aynı mantık + Markdown ## başlıklarında bölüm sınırı + açık formül ortamı içindeyken chunk sınırını bastırma. Açık formül kalırsa `[ INCOMPLETE_FORMULA ]` marker'ı eklenir. Desenler: `_HEADING_RE`, `_FORMULA_OPEN_RE`, `_FORMULA_CLOSE_RE` — `hybrid_chunker.py:1–199`.

Çıktı: `TextChunk(paper_id, chunk_index, text, page_number, section_name); chunk_id = "{paper_id}_c{chunk_index:04d}", token_estimate = char_count // 4` — `chunker.py:43–61`.

Parametreler: Varsayılan `chunk_size=1200` ve `chunk_overlap=200`, `settings.py:77–78`'de tek otoritatif kaynak olarak tanımlıdır. Bu değerler için kodda başka bir çelişen varsayılan bulunmamaktadır.

Dosya: `app/ingestion/chunker.py:23–156`, `app/ingestion/hybrid_chunker.py:1–199`, `app/config/settings.py:77–78`.

`late_chunker.py` iskelet modüldür ve şu an `HybridChunker`'a *delege* eder; büyük-bağlam embedding → alt-chunk projeksiyonu TODO durumdadır (kodda yorum olarak işaretli).

Aşama 3 — Embedding ve Vektör Depo

NE: Chunk metinlerini embedding vektörlerine dönüştürüp ChromaDB'ye yazar.

NASIL (embedding): `EmbeddingService.embed(texts)` ilk çağrıda modu belirler: Ollama erişilebilirse "ollama", değilse "fake". Ollama yolu metinleri 64'lük batch'lere böler (`_BATCH_SIZE`), `POST {host}/api/embed({"model", "input"})` çağırır; batch başarısız olursa tek tek `/api/embeddings` endpoint'ine düşer. Fake yol, SHA256 seed + counter ile `_FAKE_DIM=256` boyutlu, L2-normalize, **deterministik** vektör üretir (yalnız test/çevrimdışı; semantik değer taşımaz) — `embedding_service.py:19, 38–132`.

Embedding boyutu hakkında (dürüstlük notu): Ollama `nomic-embed-text` ile üretilen gerçek embedding'in boyutu **Ollama modeli tarafından belirlenir** ve kodda sabit bir değer olarak ayarlanmaz. Kodda görünen tek sabit boyut `_FAKE_DIM = 256`'dır (`embedding_service.py:19`) ve bu **yalnızca çevrimdışı/test için kullanılan deterministik fake yedek embedder'a** aittir. Bu nedenle 256-d, gerçek `nomic-embed-text` çıktısının boyutu olarak yorumlanmamalıdır (`nomic-embed-text`'in gerçek boyutu 256 değildir). ChromaDB collection boyutu, ilk yazılan vektörlerin boyutuna göre oluşur.

NASIL (contextual embedding, Faz P2): `build_embed_text(text, title, section, contextual)` — `contextual=True` ise embed metnine `"{title} / {section}: {text}"` ön-eki eklenir. **Önemli:** ön-ek yalnız embedding girdisinedir; ChromaDB document alanına orijinal metin yazılır, sorguya ön-ek

eklenmez (Anthropic yaklaşımı) — `paper_indexer.py:37-48`. Varsayılan `rag_contextual_embed=False` (yarı-prefix'li korpus tutarsızlık yaratır).

NASIL (ChromaDB): `PersistentClient` lazy init; collection `paper_chunks`, `metric` `hnsw:space=cosine`. `add()` aslında `upsert` yapar (idempotent). `query()` nested list sonucunu `{chunk_id, document, metadata, distance}` dict'lerine normalize eder — `chroma_store.py:21-114`.

Model/kütüphane: Ollama `nomic-embed-text` (gerçek boyut Ollama modelince belirlenir), ChromaDB (HNSW/cosine), requests, hashlib+struct (fake yedek, 256-d).

Dosya: `app/memory/embedding_service.py:1-133`, `app/memory/chroma_store.py:1-120`.

Aşama 4 — Uçtan Uca Ingestion Orkestrasyonu

NE: `PaperIndexer.ingest_one(disc, force)` tek bir keşfedilmiş PDF'i tüm hat boyunca işler; idempotenttir.

NASIL (sıra):

- Idempotency** — `get_paper_by_hash(file_hash)` varsa ve `force=False` → skip ("already ingested").
- Başlık dedup** — `find_paper_by_title(meta.title)` farklı `paper_id` döndürürse → skip ("duplicate_title"). `_normalize_title()` regex `[\W_]+` → boşluk + küçük harf; <12 karakter başlıklar dedup'tan muaf.
- Parse + metadata** — `parse_pdf` → `extract_metadata`.
- Disk** — `extracted_text/{paper_id}.txt`, `metadata/{paper_id}.json`.
- SQLite** — `upsert_paper`, `add_chunks`.
- Embedding** — `build_embed_text` ile (opsiyonel contextual) → `embedder.embed` → `chroma.add` (`documents=orijinal metin`).
- Post-processing (her biri try/except, hata ingestion'ı bloklamaz):** `FormulaExtractor` → `ConceptGraph` → `CrossPaperSynthesizer`.

Çıktı: `IngestResult(paper_id, title, n_chunks, skipped, notes)`.

Dosya: `app/memory/paper_indexer.py:1-222`.

İdempotency seviyesi	Kontrol	Davranış
File-level	<code>file_hash</code> unique	aynı bytes → skip
Paper-level	<code>_normalize_title</code> eşleşmesi	aynı başlık (farklı bytes) → skip
Formula-level	<code>formula_exists(paper_id, name)</code>	aynı formül adı → skip
Synthesis-level	<code>_example_id()</code> = SHA256(sorted formula IDs)	aynı kombinasyon → skip

Aşama 5 — Post-Ingestion: Formül / Kavram Grafiği / Sentez

5a. FormulaExtractor (formula_extractor.py:71-141): Her chunk'tan trading/finans formülleri çıkarır. LLM varsa `_EXTRACT_PROMPT` (Türkçe, `fmt="json"`, `max_tokens=1024`) ile; yoksa kural tabanlı `_KNOWN_INDICATORS` (RSI, MACD, EMA, SMA, ATR, Bollinger, Sharpe...) yedeğine düşer. Dedup: ad bazlı + `formula_exists`. Çıktı formulas tablosuna (`formula_id`, `latex`, `plain`, `category`).

5b. ConceptGraph (concept_graph.py:55-147): Formüller arası yönlü kenarlar (`extends`, `measures`, `limits`, `combines`, `opposite_of`, `requires`) LLM `_LINK_PROMPT` ile çıkarılır; `concept_links` tablosuna yazılır. LLM yoksa boş liste döner (`graceful`).

5c. CrossPaperSynthesizer (cross_paper_synthesizer.py:54-336): Formülleri kategoriye göre gruplar, geçerli (≥ 2) kategoriden ikili/üçlü kombinasyon üretir. LLM ile sentez ister; başaramazsa 8 önceden tanımlı şablondan (`_FALLBACK_TEMPLATES`, ör. `momentum×volatility` → "Volatilite-Normalize Momentum") üretir. İdempotent: `_example_id = "syn_"+SHA256(sorted formula_ids)[:24]`. Çıktı `training_examples` tablosuna (`example_type="cross_paper_synthesis"`).

Aşama 6 — Retrieval Hattı

NE: Kullanıcı sorusunu chunk listesine çevirir (`over-fetch` + `rerank` + opsiyonel hibrit/query expansion).

NASIL (üst akış — rag_answerer.py:69-106):

```
soru → RerankingRetriever.retrieve(query, top_k)
      | DenseRetrieval (Chroma, overfetch×4)
      | Hybrid BM25 adayları (opt-in)
      | rerank (heuristik | cross-encoder)
      → MultiQueryRetriever (opt-in, RagAnswerer'dan kullanılmıyor)
```

Dense (retrieval_service.py:47-65): `embed_one(query) → chroma.query(top_k) → RetrievedChunk(chunk_id, paper_id, text, page, section, title, distance)`.

Over-fetch + rerank (reranking_retriever.py:60-103): `top_k * overfetch` (varsayılan 4) aday çeker; `rag_cross_encoder=True` ise `CrossEncoderReranker`, değilse heuristik `Reranker`.

Heuristik reranker (reranker.py:86-137): Dört faktör — `final = 0.4·semantic + 0.3·keyword + 0.2·section + 0.1·formula`. `Semantic = 1 - distance/2`; `keyword = token kesişim oranı`; `section = _SECTION_PRIORITY` (`abstract=1.0`, `references=0.1`); `formula = LaTeX deseni varsa 1. tanh(final·2) / tanh(2)` normalize, azalan sıralama.

Cross-encoder (cross_encoder_reranker.py:52-92): (soru, chunk) çiftini birlikte puanlar. Model BAAI/bge-reranker-base (çok dilli, ~280MB). Model/indirme/prediction hatası → `graceful` olarak heuristik `Reranker`'a düşer. Opt-in: `ACHILLES_RAG_CROSS_ENCODER=true` + `sentence-transformers`.

BM25 (bm25_index.py:18-108): Saf Python (dependency yok), `k1=1.5`, `b=0.75`. `IDF log((N-df+0.5)/(df+0.5)+1)`. Corpus lazy olarak Chroma'dan kurulur, modül-düzey cache; chunk sayısı değişince yeniden kurulur (`bm25_corpus.py:21-62`). Chroma boş/erişilemezse `dense-only`'a düşer.

Query expansion (query_expander.py:128-215): Kural tabanlı (LLM gerektirmez). Tam/alt-dize/token eşleşmesiyle eş anlamlı sözlüğü uygular; finans sorgularına `systematic trading / quantitative approach / risk-adjusted` son ekleri ekler; min 3 / max 5 varyant. `MultiQueryRetriever`

(multi_query_retriever.py:29-59) her varyant için ayrı retrieval yapıp chunk_id'ye göre tekilleştirir.

Self-refining RAG (self_refining_rag.py:44-114): Çok turlu — bağlam kalitesinde sorun varsa (has_incomplete_formula/has_incomplete_argument) top_k += 3 ile genişletip yeniden çeker.

Kontrol bayrakları (settings.py:76-95): rag_top_k=6 (satır 76), chunk_size=1200/
chunk_overlap=200 (satır 77-78), rag_rerank=True (satır 81), rag_overfetch=4 (satır 82),
rag_hybrid=True (satır 85), rag_cross_encoder=False (satır 89), rag_contextual_embed=False (satır 95).

Aşama 7 — Bilgi Kartı + Cevaplama

Bilgi kartı (knowledge_card_builder.py): KnowledgeCard (Pydantic, satır 30-42) alanları: main_claim, methods, datasets, trading_relevance, limitations, possible_strategy_hypotheses, risk_warnings, implementation_notes. build() (138-180): makale metni (max 6000 char) → LLM (temperature=0.1, fmt="json", max_tokens=900) → _extract_json (kod çiti temizleme, akıllı tırnak düzeltme, trailing virgöl temizleme). main_claim boşsa 3000 char ile bir retry. _classify_card zorluk (0.1/0.2/0.4 +0.2) ve stage (lora_phase_1..4) atar; güven daima "draft" (insan onayı zorunlu).

RAG cevaplayıcı (rag_answerer.py:69-106): _format_context ile chunk'ları {citation} - {title}\n{text} biçimine getirir; sistem prompt rag_answer.md (iki dilli, zorunlu [paper_id:chunk_id] atıf, bölümler: Kısa Cevap / Kaynaklar / Bağlam Kalitesi / Akademik Bulgu / Formül Analizi / Trading Hipotezi / Test Planı / Riskler / Sonraki Adım). temperature=0.2. LLM offline → graceful: retrieval sonuçlarını biçimleyip döner.

Aşama 8 — Doğrulama Katmanı

Bileşen	Dosya	Yöntem (eval/exec YOK)
CitationVerifier	citation_verifier.py:42-92	Regex \[paper:chunk\] → exists (retrieval'da var mı) + supported (±80 char bağlam token örtüşmesi ≥2)
GroundingVerifier	grounding_verifier.py:69-130	Cümle bölme → token örtüşmesi: ≥5 SUPPORTED, 3-4 PARTIAL, <3 UNSUPPORTED; spekülatif marker → SPECULATIVE(0.3)
ContextSufficiency	context_sufficiency.py:47-119	ChunkQualityFlags ile: chunk yok→INSUFFICIENT; tüm formül kesik→MISSING_FORMULA_CONTINUATION; ≥3 sorunsuz→SUFFICIENT
ConfidenceScorer	confidence_scorer.py	0.25·context + 0.30·citation + 0.30·grounding + 0.15·formula - çelişki cezası → <0.40 abstain, 0.40-0.70 warn, ≥0.70 answer
AbstentionPolicy	abstention_policy.py:37-72	Yetersiz bağlam veya abstain kararı → çekimser kal

Bu katman **objektif sayısal kıyas + regex** kullanır; `eval/exec` hiçbir yerde çalışmaz (CLAUDE.md Kural 5).

Aşama 9 — rag_mastery Metriği

NE: RAG bilgi olgunluğunu tek panoda toplar.

NASIL (`rag_mastery.py:14-56`):

- `coverage = papers_with_real / n_papers × 100` (yalnız **içerik taşıyan** kartlar).
- `train_readiness = min(1.0, n_examples / 50) × 100`.
- `comprehension = scored kartların ComprehensionScore.total ortalaması`; hiç skor yoksa **None** (sahte yüksek skor üretmez).
- `mastery = 0.40·coverage + 0.30·comprehension + 0.30·train_readiness`.

`ComprehensionScorer` (`comprehension_scorer.py:47-133`): A=kart doluluk (0.30), B=RAG precision@5 (0.40), C=LLM keyword örtüşme (0.30); LLM offline → C=0.5 varsayılan (uydurmaz). LLM-free hızlı mod (`use_llm=False`) loop'larda kullanılır.

Aşama 10 — Anlama Doğrulama Merdiveni

Çekirdek ilke (MEMORY.md ile uyumlu): **anlama yüzdeyle değil SINAVLA kanıtlanır**. Tüm sınavlar çevrimdışı, deterministik (seed'li), sahte-pass üretmez (LLM yok → `skipped`).

Taban / L1 / L2 (RAG cevapları — `understanding_score.py:84-114`): `rag_answers_to_results()`:

- `requires_abstention` → **Taban (dürüstlük)**: `passed = abstention_correct`.
- cevap boş → L1/L2 `no_data` (paydaya girmez).
- cevap var → **L1 (çıkartım)**: `citation_score ≥ 0.3`; **L2 (sadakat)**: `grounding_score ≥ 0.4` ve `hallucination` yok.
- Not: bu fonksiyon RAG sistemiyle canlı entegre değil (bekleme aşamasında).

L3 — Uygulama Sınavı (`13_application.py`): Modele göstergenin kesin tanımı + seed'li sayılar verilir; model JSON dizi döndürür; `_parse_values` (yalnız JSON, `eval/exec` yok) → `np.allclose(model, ref, rtol, atol)`. `ReferenceOracle` deterministik kapanışlar üretir (`synthetic_closes, seed`). LLM yok → `skipped`.

L4 — Karşıolgu Sınavı (`14_counterfactual.py`): Periyot 2× edilir; pürüzlülük (`_roughness = ardışık diff std`) **koddan** hesaplanır; doğru yön (`artar/azalır/ayni`) koddan gelir. Model yön tahmin eder; `_normalize` TR/EN keyword eşler. `|delta| ≤ 1e-6` → `no_data`. LLM yok → `skipped`.

L5 — Kompozisyon Sınavı (`15_composition.py:67-153`): `StrategyIR` üç kapıdan geçer:

- **Math** — tüm göstergeler registry'de, periyot >1, `RSI ∈ [0,100]` kural sınırları (`_rule_bound_problems`).
- **Novelty** — ≥2 farklı gösterge tipi + daha önce görülmemiş imza.
- **Backtest** — maliyet-dahil değerlendirme `verdict == "pass"`; veri yoksa "veri yok" → `skipped`. Üçü de geçerse **candidate**, biri düşerse **rejected**.

Safe eval (safe_eval.py:44-92): Registry-dışı formüller için **whitelist AST** (+, -, *, /, **, %, //, abs/min/max/sqrt/exp/log...). __import__, attribute, subscript, lambda yasak.

UnderstandingScore (understanding_score.py:152-174): graded = passed + failed (paydaya yalnız bunlar girer); skipped/no_data rapor edilir ama paydaya **girmez**. pass_rate = passed/graded (graded=0 ise None), status = "scored" | "insufficient_data", by_level dağılımı. score_indicator_exams(seed) tüm registry üzerinde L3+L4 koşar.

ENTROPY göstergesi (indicators.py:73-84): Yönsel ikili Shannon: yukarı-hareket oranı p için $H = -(p \cdot \log_2 p + (1-p) \cdot \log_2 (1-p))$, aralık [0,1]. Registry: periyot 4, rtol/atol 1e-3.

PERMENTROPY (indicators.py:108-133): Bandt-Pompe permütasyon entropisi (order=3), $\log_2(3!)$ normalize, Lehmer/faktöriyel kodlama, stable argsort (deterministik, look-ahead yok). Registry: periyot 8.

Araştırma döngüsü (orchestrator.py:136-315): soru → SynthesisEngine.synthesize() → StrategyIR → backtest → evaluate → L5 → (pass? dur : ReflectionAgent.reflect() → IR güncelle → tekrar). SynthesisEngine prompt'u "garanti kâr" yasaklar; her öneri hipotezdir. ReflectionAgent backtest metriklerine göre tek değişiklik önerir (az işlem→RSI eşiği düşür; aşırı drawdown→eşik yükselt + EMA filtresi).

Kullanılan Loop'lar ve Otomasyon

Loop	Tetikleyici	Ne yapar	Cadence
continuous-learning (scripts/continuous-learning.sh:1-130)	Elle başlatma	kart→approve→comprehension→(3 turda bir research+synth)→synth-qa→rag-mastery	~120 sn dinle saat
auto-chain (scripts/auto-chain.sh:1-95)	Elle başlatma	7 aşama: kart→...→lora-dataset→24h eğitim döngüsü	tek koşu + 24
mac-loop (scripts/mac-loop.sh:1-107)	Elle (macOS)	kart→approve→synth-qa→lora-dataset→MLX train (300 iter); train_status.json web rozeti	5 dk tur-arası
auto_researcher (auto_researcher.py:1-105)	CLI achilles auto-research	approved kart→hipotez sorusu→tool-use seans→reward (DPO hazırlık)	soru başına

detached_launch (detached_launch.py:1-359)	Web POST /api/ training/launch veya pipeline	ensure_train_split→atomik kilit→subprocess.Popen (DETACHED) → log'a yaz	tek başlatma
auto_pipeline background_loop (auto_pipeline.py:409-423)	auto_enabled=True ise	uyku → Gate 0-8 kontrol → READY_TO_TRAIN	check_inte: dk

auto-chain eğitim çağırısı (düzeltme notu): auto-chain.sh, eğitim aşamasını bir web sunucu subprocess'i veya HTTP POST üzerinden **değil**, doğrudan CLI komutuyla bir bash döngüsü içinde çalıştırır: `uv run achilles train --run --backend peft --adapter-name achilles_auto --iterations 40` (auto-chain.sh:90). Yani her döngü turunda eğitim, betikten ayrı bir komut süreci olarak (in-process değil) başlatılır; web/POST aracılığı yoktur (auto-chain.sh:81-92).

Önemli loop notları:

- continuous-learning başında **devralma protokolü**: `touch storage/STOP_TRAINING` → 360×15sn (~90 dk) bekle → +150 sn cooldown → kendi başlar (`rm -f`).
- Otomatik arXiv çekme KAPALI** (2026-06-14 kullanıcı isteği); loop yalnız elle yüklenmiş makaleler üzerinde çalışır (continuous-learning.sh:50-54).
- Sürekli CPU-LoRA DURDURULDU**: 4B CPU'da ~76 sn/adım, 15-50 örnek overfit eder; eğitim ≥1000 örnekte bulut-GPU'da (rag-training-redesign kararı).
- ScheduleWakeup otonom nöbet kodda bulunamadı**: HANDOFF notuna göre yeni seans kendiliğinden devam etmez; loop'lar yalnız elle/CLI/web ile başlar.

Kararlar ve Dersler

- 1. Anlama yüzdeyle değil sınavla kanıtlanır.** Kaba `ComprehensionScorer` (%-tabanlı self-değerlendirme) objektif değildi; yerine L3/L4/L5 + `UnderstandingScore` eklendi — sayısal kıyas (`np.allclose`), yön referansı koddan, sahte-pass üretmez (`l3_application.py:89-100`). Bu, CLAUDE.md Kural 2'nin ("test edilmeden çalışıyor deme") doğrudan uygulamasıdır.
- 2. v5 adapter regresyonu (MEMORY.md, adapter_eval_achilles_lora_v5*.json).** v5 eğitimi bitti ama disiplinde GERİLEDİ. `adapter_eval` raporu: `base_score=-2.0`, `adapter_score=-1.0`, **verdict=accept** — yine de adapter "15 dakikalık periyotlarda..." ifadesini 5 kez tekrarladı (`degenerate_repetition`). Ders: degenerasyon cezası `score`'a yeterince ağır yansımıyor; **negasyon-kör** flag kontrolü ("kesinlikle değil" → yanlış flag) ve inference hata yönetimi **kodda bulunamadı**. Üretim öncesi manuel inceleme zorunlu.
- 3. eval/exec hiçbir yerde yok.** Hem doğrulama hem registry-dışı formül değerlendirmesi whitelist AST (`safe_eval.py`) veya yalnız-JSON parse ile yapılır (CLAUDE.md Kural 5).
- 4. Determinizm her yerde.** Dataset split `seed=42` (`detached_launch.py`), sentetik kapanışlar `seed`'li (`reference_oracle.py`), fake embedding SHA256-deterministik. (CLAUDE.md Kural 6).
- 5. Contextual embedding tutarlılık riski.** `rag_contextual_embed` varsayılan kapalı; açılırsa **tüm korpus** `reindex-contextual` ile yeniden embed edilmeli (yarı-prefix'li korpus tutarsızdır) — `main.py:1851-1911`.

6. **Idempotency çok seviyeli.** file_hash + başlık + formül adı + sentez imzası; ingestion her zaman güvenli tekrarlanabilir.
7. **Grounding ile uydurma savunması.** Sentetik-QA _is_grounded üç kapısı (sayı-altküme, bilgi-fakir reddi, anchor örtüşmesi) uydurma metrik tespitinin en güçlü savunmasıdır (CLAUDE.md Kural 7).
8. **Graceful degradation ilkesi.** LLM offline iken sistem durmaz: rule-based formül çıkarma, şablon sentez, heuristik rerank, fake embedding, "LLM offline — retrieval results only" cevabı.

Dosya Referans Haritası

Dosya	Görev
app/ingestion/pdf_parser.py	PDF→metin (PyMuPDF/pypdf)
app/ingestion/chunker.py	Matematik-farkında temel chunking
app/ingestion/hybrid_chunker.py	Başlık + formül-ortamı koruyan chunking
app/ingestion/arxiv_fetcher.py	arXiv arama + PDF indirme (loop'ta otomatik çağrı KAPALI)
app/ingestion/metadata_extractor.py	Başlık/yazar/yıl sezgisel çıkarımı
app/memory/paper_indexer.py	Uçtan uca ingestion orkestrasyonu + idempotency
app/memory/embedding_service.py	Ollama nomic-embed-text + fake fallback (256-d, yalnız test/çevrimdışı)
app/memory/chroma_store.py	ChromaDB vektör depo (cosine/HNSW)
app/memory/sqlite_store.py	İlişkisel depo (papers/chunks/formulas/cards/sessions)
app/memory/retrieval_service.py	Dense semantic retrieval
app/memory/reranking_retriever.py	Over-fetch + rerank sarmalayıcı
app/memory/reranker.py	4 faktörlü heuristik rerank
app/memory/cross_encoder_reranker.py	Model tabanlı rerank (opt-in, graceful)
app/memory/hybrid_retriever.py	Alpha-harman semantic+BM25
app/memory/bm25_index.py / bm25_corpus.py	Saf Python BM25 + lazy corpus
app/memory/contextual_chunker.py	Chunk kalite bayrakları (ingestion'da henüz tetiklenmiyor)
app/brain/knowledge_card_builder.py	Bilgi kartı üretimi (Pydantic + LLM JSON)
app/brain/rag_answerer.py	RAG cevap üretimi + graceful degrade
app/brain/query_expander.py	Kural tabanlı query genişletme
app/brain/multi_query_retriever.py	Çoklu sorgu birleştirme (opt-in)

app/brain/self_refining_rag.py	İteratif kalite kontrol
app/prompts/rag_answer.md	İki dilli sistem prompt (atıf + format)
app/verification/ citation_verifier.py	[paper:chunk] atıf doğrulama
app/verification/ grounding_verifier.py	Cümle-chunk dayanak doğrulama
app/verification/ context_sufficiency.py	Bağlam yeterliliği sınıflandırma
app/verification/ confidence_scorer.py	Ağırlıklı güven + answer/warn/abstain
app/verification/ abstention_policy.py	Çekimserlik kararı
app/verification/ comprehension_scorer.py	A/B/C kaba anlama skoru
app/verification/rag_mastery.py	coverage+comprehension+train_readiness bileşik
app/verification/exams/ l3_application.py	L3 sayısal uygulama (np.allclose)
app/verification/exams/ l4_counterfactual.py	L4 karşıolgu yön
app/verification/exams/ l5_composition.py	L5 math+novelty+backtest kapıları
app/verification/exams/ understanding_score.py	Sınav agregasyonu + RAG→sonuç adaptörü
app/verification/exams/registry.py	SMA/EMA/RSI/ENTROPY/PERMENTROPY ExamSpec'leri
app/verification/exams/ reference_oracle.py	Seed'li sentetik kapanışlar
app/verification/exams/safe_eval.py	Whitelist AST (eval/exec yok)
app/trading/indicators.py	ENTROPY + PERMENTROPY (ve diğer göstergeler)
app/research/formula_extractor.py	Formül çıkarımı (LLM/kural)
app/research/concept_graph.py	Kavram grafiği kenarları
app/research/ cross_paper_synthesizer.py	Çapraz makale sentezi
app/research/orchestrator.py	Araştırma yaşam döngüsü
app/research/synthesis_engine.py	Hipotez/StrategyIR üretimi
app/research/reflection_agent.py	Backtest yansıması
app/learning/rag_exam_runner.py	RAG sınav koşucusu

scripts/continuous-learning.sh	72h öğrenme döngüsü
scripts/auto-chain.sh	Tek-seferde tam zincir
scripts/mac-loop.sh	macOS MLX eğitim döngüsü
app/pipeline/auto_researcher.py	kart→soru→tool-use→reward
app/training/detached_launch.py	Detached eğitim başlatma + durum
app/config/settings.py	RAG hiperparametreleri (chunk_size/overlap satır 77-78)

Test Kapsamı

Proje testleri `tests/` kökünde toplanmıştır (yaklaşık 70 test dosyası). Aşağıdaki katmanlar burada test edilir:

- **Doğrulama / sınav (L3/L4/L5/UnderstandingScore):** `test_l3_application.py`, `test_l4_counterfactual.py`, `test_l5_composition.py`, `test_understanding_score.py`, `test_citation_verifier.py`, `test_context_sufficiency.py`, `test_abstention_policy.py`, `test_reference_oracle.py`, `test_safe_eval.py`, `test_cli_exams.py`.
- **Retrieval (BM25 + hibrit + cross-encoder + rerank):** `test_bm25_index.py`, `test_hybrid_retrieval.py`, `test_cross_encoder_reranker.py`, `test_reranker.py`, `test_reranking_retriever.py`, `test_multi_query_retriever.py`, `test_query_expander.py`.
- **Embedding / depo:** `test_embedding_and_chroma.py`, `test_contextual_embed.py`.
- **Göstergeler:** `test_entropy_indicator.py`, `test_permutation_entropy.py`, `test_indicators.py`.

Not: `app/verification/tests/` adlı bir **alt-dizin yoktur**; doğrulama ve merdiven testleri ayrı bir alt-dizin yerine ortak `tests/` kökünde tutulur. Bu doküman, bu testlerin **var olduğunu** belgeler; testlerin güncel çalışma durumu (geçti/kaldı) bu dokümanda iddia edilmez — kanıt için `make test` çalıştırılmalıdır (CLAUDE.md Kural 2).

Bilinen Sınırlamalar

1. **Contextual chunker entegre değil.** `contextual_chunker.py` kalite bayrakları yazılmış ama ingestion'da tetiklenmiyor; `chunk_quality_flags` tablosu dolmuyor.
2. **MultiQueryRetriever bağlı değil.** Yazılı ama `RagAnswerer` doğrudan retrieve kullanıyor.
3. **reindex-contextual otomasyonu yok.** Elle çalıştırılır; scheduler/cron kodda bulunamadı.
4. **BM25/cross-encoder testleri mevcut (kapsam derinliği ayrıca ölçülmedi).** Birim/entegrasyon düzeyinde testler vardır: `test_bm25_index.py`, `test_hybrid_retrieval.py`, `test_cross_encoder_reranker.py`, `test_reranking_retriever.py`. Uçtan uca (canlı Ollama + canlı model indirme dahil) tam senaryo kapsamının derinliği bu dokümanda ayrıca ölçülmemiştir; ancak "test edilip edilmediği belirsiz" değildir — testler mevcuttur.
5. **rag_answers_to_results canlı RAG'a bağlı değil** (bekleme aşamasında).
6. **adapter_eval zayıflıkları:** degenerasyon cezası hafif; negasyon-kör flag kontrolü ve inference

hata yönetimi kodda bulunamadı; v5 degenerate çıktıya rağmen "accept" verdi → manuel inceleme şart.

7. **Bağlam yetersiz dilim:** halüsinasyon algılama yalnız UNSUPPORTED cümleleri yakalar; kısmi-doğru/yanlış karışımı token örtüşmesiyle maskelenebilir.
8. **chunk_size varsayılanı nettir.** settings.py:77'de chunk_size=1200 ve settings.py:78'de chunk_overlap=200 olarak tek otoritatif kaynakta tanımlıdır; kodda çelişen bir değer (ör. "~8000") bulunmamaktadır.
9. **L3/L4 küçük veri seti gürültüsü:** varsayılan nokta sayıları düşük; az örnekte gürültü yüksek olabilir.
10. **app/verification/tests/ alt-dizini yok:** doğrulama ve merdiven testleri ayrı bir alt-dizinde değil, ortak tests/ kökünde toplanmıştır (bkz. "Test Kapsamı"). Eksik olan tek şey bu beklenen alt-dizindir; testlerin kendileri mevcuttur.

Sözlük

- **RAG** — Retrieval-Augmented Generation: soruya, vektör deposundan çekilen ilgili pasajlarla cevap üretme.
- **Chunk** — bir makalenin parçası; chunk_id = paper_id_cNNNN. Matematik blokları bölünmeden korunur.
- **Embedding** — metnin sayısal vektör temsili. Birincil yol Ollama nomic-embed-text (gerçek boyut Ollama modeli tarafından belirlenir; kodda 256 olarak sabitlenmez). 256-d **yalnızca** çevrimdışı/test için kullanılan deterministik fake yedek embedder'a aittir (embedding_service.py:19, _FAKE_DIM=256). Cosine mesafesiyle aranır.
- **Contextual retrieval (P2)** — embed metnine "başlık / bölüm:" ön-eki ekleme; ChromaDB document'ı orijinal kalır.
- **Over-fetch** — top_k × overfetch aday çekip rerank sonrası top_k'ya indirme.
- **BM25** — kelime sıklığı tabanlı klasik sıralama; teknik terimleri dense'in kaçırdığı yerde yakalar.
- **Cross-encoder** — (soru, chunk) çiftini birlikte puanlayan ağır ama doğru reranker.
- **Citation/Grounding** — atıfların gerçekten var olup olmadığı / cümlelerin chunk'larca desteklenip desteklenmediği.
- **Abstention (çekimserlik)** — güven düşük veya bağlam yetersizse cevap vermeme; doğru davranış sınanır (Taban).
- **rag_mastery** — coverage(%40)+comprehension(%30)+train_readiness(%30) bileşik bilgi-olgunluk skoru.
- **UnderstandingScore** — L1–L5 sınav sonuçlarından objektif pass_rate; skipped/no_data paydaya girmez.
- **L3/L4/L5** — Uygulama (np.allclose) / Karşıolgu (yön) / Kompozisyon (math+novelty+backtest) sınavları.
- **ENTROPY** — yönsel ikili Shannon entropisi; 0.5 oran → 1 (maks belirsizlik), net trend → 0.
- **PERMENTROPY** — Bandt-Pompe permütasyon (ordinal) entropisi; order=3, [0,1] normalize.
- **StrategyIR** — strateji ara temsili (göstergeler + entry/exit kuralları); backtest+evaluate girdisi.
- **Verdict** — backtest sonrası pass/fail/inconclusive; pass değilse çıktı "aday"dır, "hazır" değil.
- **Idempotency** — aynı girdiyi tekrar işlemenin yan etki üretmemesi (file_hash/başlık/formül/sentez seviyelerinde).
- **Detached eğitim** — web/terminal kapansa da süren OS subprocess'i; ilerleme log tazeliğinden

okunur.

- **Graceful degradation** — LLM/model yokken kural-tabanlı yedeklerle sistemin durmadan çalışmaya devam etmesi.